

SBOMGuard – Research Paper Summary

1. An Empirical Study on Software Bill of Materials: Where We Stand and the Road Ahead

Authors: Boming Xia, Tingting Bi, Zhenchang Xing, Qinghua Lu, and Liming Zhu.

Paper Summary:

This paper presents an empirical study of how SBOMs are understood and used by software practitioners. It investigates practical issues related to SBOM adoption, generation, distribution, validation, consumption, maintenance, tooling, standards, and security. The study uses practitioner interviews and a survey to understand current SBOM practices and the challenges faced by organizations.

Research Gap:

Earlier work mainly discussed the benefits, standards, and adoption of SBOMs, but there was limited empirical evidence about how practitioners actually use and maintain them. This study addresses that gap by examining real-world practitioner experiences and identifying practical challenges in SBOM adoption, tooling, standardization, integrity, and vulnerability management.

2. On the Way to SBOMs: Investigating Design Issues and Solutions in Practice

Authors: Tingting Bi, Boming Xia, Zhenchang Xing, Qinghua Lu, and Liming Zhu.

Paper Summary:

This study investigates practical SBOM-related problems found in software-development environments. Instead of focusing only on surveys, it analyzes real-world developer discussions to identify SBOM design issues, implementation challenges, and possible solutions across the software lifecycle. It provides a practitioner-oriented view of how SBOMs are developed, integrated, and maintained.

Research Gap:

Previous work examined SBOM adoption and practitioner opinions, but less research investigated concrete design problems arising during real development activities. This paper addresses the gap by analyzing real software-repository discussions and identifying practical SBOM problems and solutions.

3. SBOM Generation Tools in the Python Ecosystem: an In-Detail Analysis

Authors: Serena Cofano, Giacomo Benedetti, and Matteo Dell’Amico.

Paper Summary:

This paper provides a detailed investigation of SBOM-generation tools in the Python ecosystem. It studies how tools handle package managers, metadata files, dependencies, transitive dependencies, and different Python project configurations. The work focuses on why SBOM generators can produce incomplete or inconsistent results and examines the ecosystem-specific causes of these problems.

Research Gap:

Earlier studies compared SBOM generators across multiple languages but provided less detail about why differences occur inside a particular ecosystem. This work addresses the gap by deeply analyzing Python package-management and dependency-resolution behavior and identifying the causes of SBOM inaccuracies.

4. The Impact of SBOM Generators on Vulnerability Assessment in Python: A Comparison and a Novel Approach

Authors: Giacomo Benedetti, Serena Cofano, Alessandro Brighente, and Mauro Conti.

Paper Summary:

This paper investigates how the choice of SBOM generator affects vulnerability assessment in Python applications. It compares several SBOM-generation approaches and evaluates how accurately they identify dependencies that are later used for vulnerability analysis. The study also proposes a package-manager-oriented approach intended to improve the quality of vulnerability assessment.

Research Gap:

Earlier research focused mainly on SBOM-generation correctness, while less attention was given to the effect of incorrect or incomplete SBOMs on downstream vulnerability analysis. This paper connects SBOM-generation quality with practical vulnerability-assessment accuracy.

5. A Reality Check on SBOM-based Vulnerability Management: An Empirical Study and A Path Forward

Authors: Li Zhou, Marc Dacier, and Charalambos Konstantinou.

Paper Summary:

This paper evaluates SBOM-based vulnerability management using a large collection of open-source repositories. It examines whether lock files can provide accurate dependency information and investigates the large number of false-positive vulnerability reports produced by package-level scanning. The study combines accurate SBOM generation with function-level reachability analysis to reduce irrelevant vulnerability findings.

Research Gap:

Existing SBOM vulnerability scanners often report a vulnerability when a vulnerable package exists, even when the vulnerable functionality is not actually reachable. This paper addresses the gap by combining accurate dependency information with reachability analysis to improve vulnerability-report accuracy.

6. On the Security Blind Spots of Software Composition Analysis Caused by Cloning and Shading

Authors: Jens Dietrich, Shawn Rasheed, Alexander Jordan, and Tim White.

Paper Summary:

This paper studies security weaknesses in conventional Software Composition Analysis when vulnerable code is copied, cloned, repackaged, or hidden through shading. Traditional SCA tools commonly depend on package names and version information, so they may fail to recognize vulnerable code after its original identity has been changed. The study demonstrates how these hidden forms of reuse can create security blind spots.

Research Gap:

Existing dependency-based SCA approaches are effective when the original

package identity is preserved but can miss vulnerable code that has been copied or shaded. This paper addresses that gap by examining vulnerability detection at the code level rather than relying only on package metadata.

7. Hidden Dependencies and Component Variants in SBOM-Based Software Composition Analysis

Authors: Shawn Rasheed, Max McPhee, Lisa Patterson, Stephen MacDonell, and Jens Dietrich.

Paper Summary:

This paper examines cases where SBOM component information does not fully represent the actual code-level dependencies of an application. It focuses on hidden dependencies and component variants and investigates how these mismatches affect vulnerability analysis and VEX-based decisions. The study extends code-level dependency analysis beyond simple component matching.

Research Gap:

Existing SBOM and SCA approaches can fail when code-level dependencies differ from the component-level relationships represented in an SBOM. The paper addresses this gap by studying hidden dependencies, component variants, and the effect they have on vulnerability interpretation and suppression.

8. A Landscape Study of Open Source and Proprietary Tools for Software Bill of Materials (SBOM)

Authors: Mehdi Mirakhorli, Derek Garcia, Schuyler Dillon, Kevin Laporte, Matthew Morrison, Henry Lu, Viktoria Koscinski, and Christopher Enoch.

Paper Summary:

This paper provides a large empirical study of the SBOM tooling ecosystem. It analyzes **84 open-source and proprietary tools** and classifies them into use cases such as SBOM generation, consumption, interoperability, quality assurance, and SBOM services. The study also conducts a Plugfest to compare SBOM-generation tools and identify differences in generated information, dependency relationships, schemas, and interoperability.

Research Gap:

The SBOM ecosystem had many tools but lacked a clear classification of tools and use cases, along with consistent approaches for quality and interoperability.

This study addresses those gaps through a broad tooling landscape analysis and practical benchmarking.

9. Vulnerability Analysis Based on Software Bill of Materials (SBOM): A Model Proposal for Automated Vulnerability Scanning for CI/CD Pipelines

Authors: Ömercan Kağızmandere and Halil Arslan.

Paper Summary:

This paper proposes an automated CI/CD model that generates an SBOM from source code and then scans the SBOM for vulnerabilities in software dependencies. The researchers evaluated 10 popular C# open-source projects using Microsoft SBOM Tool and Bomber. Although the main projects themselves had no recorded vulnerabilities in the MITRE-CVE system, the dependency-level SBOM analysis revealed vulnerabilities in their components. The authors recommend integrating SBOM generation and vulnerability scanning into CI/CD pipelines.

Research Gap:

Traditional application-level vulnerability checks can overlook vulnerabilities inside dependencies. The paper addresses this gap by integrating automated SBOM generation and dependency vulnerability analysis into CI/CD workflows.

10. Distributed Software Build Assurance for Software Supply Chain Integrity

Authors: Ken Lew, Arijet Sarker, Simeon Wuthier, Jinoh Kim, Jonghyun Kim, and Sang-Yoon Chang.

Paper Summary:

This paper proposes a distributed software-build assurance mechanism combining **SBOMs, reproducible builds, cryptographic hashes, Merkle trees, and blockchain**. The system creates a proof from source code, build output, and SBOM documentation and compares it with a trusted reference stored on a permissioned blockchain. The approach was implemented using Ethereum software and demonstrated the ability to detect even very small modifications to software artifacts.

Research Gap:

Existing techniques generally address individual aspects of software integrity, such as remote attestation, reproducible builds, or SBOM documentation. This paper combines these techniques to verify multiple software artifacts across the supply chain and removes dependence on a centralized verification authority.

11. Risk-Aware Software Supply Chain Security For Aviation Systems Using SBOM

Authors: Amelia Hart, Oliver Bennett, and James Whitfield.

Paper Summary:

This paper proposes an SBOM-driven security mechanism for aviation-oriented multi-cloud DevSecOps environments. It combines CycloneDX SBOMs, vulnerability information from OSV, and rule-based blocking to prevent container images containing unresolved high-risk components from reaching OTA staging. The evaluation uses **128 microservices across three cloud providers** and measures vulnerability exposure time, unsafe deployments, and OTA gate accuracy.

Research Gap:

Most SBOM and DevSecOps research has focused on general cloud environments rather than aviation-related, multi-cloud, OTA systems. Existing aviation research also provides limited empirical evidence on integrating SBOMs, vulnerability intelligence, and release-policy enforcement. This paper addresses that gap through a multi-cloud aviation-oriented evaluation.

12. An Intelligent SBOM-Driven Framework for Adaptive Vulnerability Risk Scoring in Cloud Supply-Chain Security

Authors: Hafiz Mahdi Hasan and Md. Aziz Hosen Foysal.

Paper Summary:

This paper proposes a lightweight automated framework that combines **Syft** for SBOM generation with **Grype** for vulnerability scanning. The detected vulnerabilities are assigned weighted scores based on severity and then converted into a normalized 0–100 risk score. The framework is evaluated before and after dependency patching to demonstrate how remediation changes the calculated risk. The paper also discusses future integration of EPSS, runtime context, container analysis, and multi-cloud environments.

Research Gap:

Existing vulnerability scanners can generate large numbers of findings but provide limited prioritization. This paper addresses the need for a simpler quantitative risk metric while identifying limitations of severity-only scoring, particularly the lack of exploitability and runtime context.

13. AI-Driven SBOM: Automated Software Bill of Materials Generation and Management

Author: Osha Shukla.

Paper Summary:

This paper proposes an AI-based SBOM generation and management framework combining **NLP, graph neural networks, and CNN-based binary analysis**. The system processes source code, package information, and binaries to identify components, predict transitive dependencies, detect embedded libraries, map vulnerabilities, and identify licenses. It generates SBOMs in SPDX and CycloneDX formats and evaluates the approach against existing SBOM tools.

Research Gap:

Earlier SBOM systems often depended on a single analysis method and had difficulty with dynamic dependencies, binary components, continuous updates, and license compliance. The paper addresses these gaps through a multi-modal AI architecture.

14. Supply Chain Risk Analysis via SBOM Data Enrichment

Authors: Antoine Lemay and Neeraj Katiyar.

Paper Summary:

This paper investigates whether SBOM data can be enriched with repository-maintenance information to identify libraries that may be susceptible to XZ-type supply-chain infiltration. The researchers combine an organization's SBOM inventory with GitHub and npm metadata such as contributor count, commits, last-update date, repository size, and maintenance activity. They develop a risk metric to identify libraries showing multiple risk indicators and investigate several high-risk examples.

Research Gap:

Earlier SBOM research concentrated mainly on adoption, generation, vulnerability detection, and licensing. The authors found little research using SBOM data analytics to study provenance, maintenance behavior, developer behavior, or enriched supply-chain risk. This paper addresses that gap by combining SBOM data with external repository metadata.